

Understanding Transformers

A Step-by-Step Mathematical Guide to Building a Transformer from
Scratch in PyTorch

Generated Educational Material

Contents

1	Introduction to the Transformer	1
2	Data Preparation & Tokenization	2
2.1	The Vocabulary	2
2.2	Tokenization Pipeline	2
2.2.1	A Quirky Edge Case in the Code	2
3	The Embedding Space	4
3.1	Word Embedding	4
3.2	Positional Encoding	4
3.2.1	Applying the Dropout Factor	4
3.3	Segment Embeddings	5
4	The Transformer Encoder Layer	6
4.1	Multi-Head Self-Attention Mechanism	6
4.1.1	1. Creating Q, K, and V	6
4.1.2	2. Splitting into Heads	6
4.1.3	3. Calculating Attention Scores	6
4.1.4	4. Applying the Mask & Softmax	6
4.1.5	5. Weighting the Values and Merging	7
4.2	Add & Norm, and Feed Forward	7
5	The Transformer Decoder Layer	8
5.1	Masked Multi-Head Self-Attention	8
5.2	Encoder-Decoder Cross-Attention	8
5.3	Feed-Forward and Output	8
6	The Full Transformer Assembly	9
6.1	The Final Projection	9

Chapter 1

Introduction to the Transformer

The Transformer architecture revolutionized artificial intelligence by introducing the **Self-Attention** mechanism, removing the need for sequential data processing found in older RNNs (Recurrent Neural Networks).

This textbook is a complete, beginner-friendly walkthrough of a PyTorch implementation of the Full Transformer (Encoder and Decoder). We will not only look at the Python code but also dissect the underlying mathematics, tracing how a specific input sentence—“anugrah is learning”—is converted into numerical vectors and passed, layer by layer, through the entire network.

Mathematical Insight: The Blueprint

A standard Transformer consists of:

- **Tokenization & Embedding:** Converting text to numerical vectors.
- **Positional Encoding:** Giving the model a sense of word order.
- **Encoder Layers:** Multi-Head Self-Attention + Feed-Forward Networks.
- **Decoder Layers:** Masked Self-Attention + Encoder-Decoder Cross-Attention + Feed-Forward Networks.
- **Linear Output:** Projecting final vectors back into vocabulary probabilities.

Chapter 2

Data Preparation & Tokenization

Before a neural network can process text, the text must be translated into numbers. This is done via a *Vocabulary* mapping.

2.1 The Vocabulary

In our implementation, we define a vocabulary of 39 distinct tokens. Special tokens dictate the structure of the sequences:

- `<pad>` (ID 0): Padding token, used to make all sentences the same length.
- `<unk>` (ID 1): Unknown token, for words not in the dictionary.
- `<cls>` (ID 2): Classification or Start-of-Sequence token.
- `[sep]` (ID 3): Separator token, used between different sentences.

2.2 Tokenization Pipeline

Let's analyze the input: "anugrah is learning". We want to format this into a sequence of exactly 10 tokens (our chosen `max_seq_len = 10`).

Code Breakdown: The Modular Tokenization Functions

The pipeline consists of specific helper functions:

1. `_add_special_tokens_and_segments`: Wraps the input in special tokens. Expected output format: `<cls> word1 word2 ... [sep]`.
2. `_convert_to_ids`: Maps strings to integers using the vocabulary.
3. `_pad_ids_and_segments`: Appends `<pad>` tokens until the sequence reaches length 10.
4. `_create_attention_mask_from_ids`: Creates a binary mask (1 for real tokens, 0 for padding) so the network ignores empty space.

2.2.1 A Quirky Edge Case in the Code

If we look closely at the output in the Jupyter Notebook:

```
1 Input Text: 'anugrah is learning'
2 Input IDs: tensor([[ 1, 38,  5,  9,  1,  0,  0,  0,  0,  0]])
```

Notice the sequence starts with 1 and ends with 1 before the zeros. Why 1 (`<unk>`) instead of 2 (`<cls>`) and 3 (`[sep]`)? In the helper function:

```
1 tokens = [vocab.get("<cls>", "<cls>")] # Starts with string "<cls>"
```

Because `vocab.get` searches for '`<cls>`' and finds it (value 2), it mistakenly returns the integer 2 during string construction, or if it doesn't match string types perfectly in the ID mapping, it falls back to the default `<unk>` value which is 1. This is a common bug in custom tokenizers! The model processes it smoothly anyway as IDs `[1, 38, 5, 9, 1, 0, 0, 0, 0, 0]`.

Vector Tracking: Output of Data Preparation

- **Input IDs Shape:** `[1, 10]` (Batch Size 1, Sequence Length 10)
- **Input IDs:** `[1, 38, 5, 9, 1, 0, 0, 0, 0, 0]`
- **Attention Mask Shape:** `[1, 1, 1, 10]`
- **Attention Mask:** `[[[1, 1, 1, 1, 1, 0, 0, 0, 0, 0]]]`
- **Token Type IDs Shape:** `[1, 10]` (All 0s, as there is only one sentence segment).

Chapter 3

The Embedding Space

A raw ID (like 38 for "anugrah") has no mathematical relationship to other numbers. We must map these IDs into continuous, dense vectors. In our model, we set the embedding dimension to $d_{model} = 8$.

3.1 Word Embedding

The layer `nn.Embedding(39, 8)` acts as a lookup table matrix of size 39×8 .

When we input the ID 38 (anugrah), it retrieves a specific 8-dimensional vector:

$$E_{word}(\text{anugrah}) = [0.1271, -1.5969, 1.5901, 0.3746, 0.9116, -0.6469, -0.4991, 0.5582]$$

3.2 Positional Encoding

Since Transformers process all words simultaneously (in parallel), they have no concept of word order. We inject order by adding a *Positional Encoding* vector to the word vector.

Mathematical Insight: Positional Encoding Formulas

For a given position pos in the sequence, and a given dimension index i :

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

For the word "anugrah", the position is $pos = 1$ (since position 0 is the starting token). For $d_{model} = 8$, i ranges from 0 to 3. Calculations for $pos = 1$:

- $i = 0$: $\sin(1/10000^0) = \sin(1) \approx 0.8415$, $\cos(1) \approx 0.5403$
- $i = 1$: $\sin(1/10000^{2/8}) = \sin(1/10) \approx 0.0998$, $\cos(1/10) \approx 0.9950$

$$PE(pos = 1) \approx [0.8415, 0.5403, 0.0998, 0.9950, \dots]$$

3.2.1 Applying the Dropout Factor

In the PyTorch code, after adding the positional encoding, a Dropout layer is applied: `self.dropout(x)` with probability $p = 0.1$. During training, PyTorch scales the remaining non-dropped values by $\frac{1}{1-p} = \frac{1}{0.9} \approx 1.1111$ to maintain the expected sum of the vectors.

Therefore, the combined vector output in the notebook for "anugrah" is actually:

$$E_{pos_added} = (E_{word} + PE) \times 1.1111$$

Notebook Output for anugrah (Combined word + pos): `[0.0, 0.0, 1.8777, 1.5218, 1.0240, 0.3922, 0.0, 0.0]`
(Note the zeros represent values that were dropped out and set to 0 by the layer!)

3.3 Segment Embeddings

Finally, we add Segment Embeddings (from `token_type_embedding`), which distinguish Sentence A from Sentence B. Since all our `token_type_ids` are 0, we add the exact same segment vector to every word.

Vector Tracking: Final Transformer Input

$$X = E_{pos_added} + E_{segment}$$

Shape entering the Encoder: **[1, 10, 8]**

Chapter 4

The Transformer Encoder Layer

The Encoder's job is to read the sequence and build a deep, contextual representation of every token.

4.1 Multi-Head Self-Attention Mechanism

Mathematical Insight: The Core Attention Equation

Attention calculates how much each word should "focus" on every other word in the sentence.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Where: Q (Query), K (Key), and V (Value) are linear projections of the input X .

4.1.1 1. Creating Q, K, and V

The input X has shape $(1, 10, 8)$. We multiply X by learnable weight matrices $W_q, W_k, W_v \in \mathbb{R}^{8 \times 8}$.

$$Q = XW_q \implies \text{Shape: } (1, 10, 8)$$

(The same applies for K and V).

4.1.2 2. Splitting into Heads

To allow the model to focus on different aspects simultaneously, we split the 8 dimensions into `num_heads=2`. Each head gets `head_dim = 8 / 2 = 4` dimensions. The new shape for Q, K, V is: **(1, 2, 10, 4)** (Batch = 1, Heads = 2, SeqLen = 10, Dim = 4).

4.1.3 3. Calculating Attention Scores

For each head, we compute the dot product between Queries and Keys:

$$\text{Scores} = \frac{QK^T}{\sqrt{4}}$$

Since Q is (10×4) and K^T is (4×10) , the result is an attention matrix of size **(10 × 10)**. The element at row i , column j tells us how much word i attends to word j .

4.1.4 4. Applying the Mask & Softmax

Our padding mask prevents the model from paying attention to the `<pad>` tokens. The mask sets all columns corresponding to `<pad>` to $-\infty$.

$$\text{Scores} = \begin{bmatrix} 1.2 & 0.4 & -1.1 & 0.8 & 0.1 & -\infty & \dots \\ \dots & \dots & \dots & \dots & \dots & -\infty & \dots \end{bmatrix}$$

When we apply `softmax()`, the $-\infty$ values become 0. The rows now sum to 1.

4.1.5 5. Weighting the Values and Merging

We multiply the (10×10) attention weights by the V matrix (10×4) to get a new (10×4) representation for each head. We concatenate the 2 heads back together to restore the shape to $(1, 10, 8)$. Finally, we pass it through an output linear layer W_o to get the final Multi-Head Attention (MHA) output.

4.2 Add & Norm, and Feed Forward

The Transformer utilizes Residual Connections (Add) and Layer Normalization (Norm) to stabilize training.

```
1 x = self.layer_norm1(x + final_mha_output)
```

Next, this output passes through a Feed Forward Network (FFN) applied independently to each position:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

In our code, W_1 expands the dimension from 8 to `ffn_hidden_dim = 32`, and W_2 projects it back to 8. This acts as a feature expansion/contraction phase. Another Add & Norm step follows. The output of the Encoder layer is $[1, 10, 8]$.

Chapter 5

The Transformer Decoder Layer

The Decoder generates the output sequence one token at a time (autoregressively).

5.1 Masked Multi-Head Self-Attention

The first sub-layer in the Decoder is identical to the Encoder's self-attention, with one critical difference: The Causal Mask (`tgt_mask`).

During training, we pass the entire target sequence to the decoder at once. However, the model shouldn't "cheat" by looking at future words to predict the current word. We apply a lower-triangular mask:

$$\text{Causal Mask} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Here, 0 will be replaced by $-\infty$ before the softmax. Word 1 can only attend to word 1. Word 2 can attend to words 1 and 2, etc.

5.2 Encoder-Decoder Cross-Attention

In the second sub-layer, the Decoder incorporates information from the Encoder.

- **Queries (Q):** Come from the Decoder's previous sub-layer output. "What am I looking for?"
- **Keys (K) and Values (V):** Come from the Encoder's Output. "Here is the context."

The calculation $\frac{QK^T}{\sqrt{d_k}}$ computes alignment between the generated target sequence so far and the entire input sequence.

5.3 Feed-Forward and Output

Just like the Encoder, the Decoder ends with an FFN and Add & Norm layer. Shape leaving the Decoder: [1, 10, 8].

Chapter 6

The Full Transformer Assembly

Finally, we instantiate the ‘FullTransformer’ class, connecting all the pieces.

```
1 class FullTransformer(nn.Module):
2     def __init__(...):
3         self.encoder = TransformerEncoder(...)
4         self.decoder = TransformerDecoder(...)
5         self.output_linear = nn.Linear(d_model, tgt_vocab_size)
```

6.1 The Final Projection

The Decoder outputs a tensor of shape $[\text{Batch}, \text{SeqLen}, d_{\text{model}}] \Rightarrow [1, 10, 8]$.

To convert these 8-dimensional vectors back into word predictions, we pass them through a final Linear layer:

$$\text{Logits} = X_{\text{decoder_out}} W_{\text{out}}$$

Where W_{out} is a matrix of shape (8×39) (since our vocabulary size is 39).

Vector Tracking: Final Output Shape

Matrix Multiplication: $(1 \times 10 \times 8) \times (8 \times 39) \Rightarrow (1 \times 10 \times 39)$

For each of the 10 positions in our sequence, we now have 39 raw scores (logits). We can apply a softmax over the 39 values to get a probability distribution and pick the word with the highest probability as the model’s prediction!

Congratulations!

You have traced a sequence from strings to integer IDs, mapped them into a continuous 8D vector space, added positional context, mixed the context across sequences using Multi-Head Self-Attention, processed the context in a Feed-Forward Network, transferred it to a Decoder, and finally projected it back into a 39-dimensional vocabulary distribution. This is the complete mathematical journey of the Transformer!